

STEAM REVIEW ANALYSIS

**A distributed analysis of 27,075 games and 32.8 million
aggregate review outcomes**

Big Data - Final Evaluation Project

Steam Insight Lab

Student 1 & Student 2

Submission date: 2026-06-24

Technology stack

HDFS / PySpark / Spark SQL / Hive Metastore / Python Visualization

Contents

01	Executive summary and dataset
02	Storage architecture and access strategy
03	Preprocessing and data quality
04	Analytical questions and methods
05	Findings and visual interpretation
06	Methodological depth and limitations
07	Conclusion, reflection and team roles
08	Reproducibility, rubric map and references

Executive summary

27,075 GAMES	32,803,682 REVIEW OUTCOMES	82.58% WEIGHTED POSITIVE	36 MEDIAN REVIEWS/GAME
------------------------	--------------------------------------	------------------------------------	----------------------------------

This project studies how Steam's catalogue, prices, genres, publishers and review outcomes relate. The analysis uses the Kaggle Steam Store Games snapshot, stores immutable CSV inputs in HDFS, converts cleaned records to year-partitioned Parquet, and answers seven questions with PySpark and Spark SQL. The key methodological choice is a 95% Wilson lower confidence bound: it ranks strongly reviewed games without rewarding tiny samples that happen to be 100% positive.

Across 27,075 games, the weighted positive share is 82.58%, while the median game-level positive rate is only 76.03%. This 6.55-point gap shows that review-volume weighting and 'typical game' summaries answer different questions. Price also has no monotonic relationship with satisfaction: the GBP 10-20 band has the highest mean positive rate, but causal claims are not justified because genre, age, visibility and audience selection are confounders.

1. Dataset introduction

The selected topic is **Steam Review Analysis**. The source is Nik Davis's Steam Store Games dataset on Kaggle, a static catalogue snapshot containing games available on Steam around May 2019. The primary table has one row per app ID and combines identity, release date, developer/publisher, multi-label platform/category/genre/tag fields, price, playtime, owner-range estimates and aggregate positive/negative recommendation counts. The accompanying SteamSpy table contains tag vote counts.

Important scope note: despite the assignment topic's wording, this dataset does not contain review text or individual reviewers. Therefore this is review-outcome and market-structure analysis, not NLP sentiment analysis. Stating that boundary prevents overclaiming and makes the unit of analysis explicit.

Core data dictionary

Field group	Examples	Analytical use
Identity	appid, name	Primary key and labels
Time	release_date	Release-year trends and cohorts
Actors	developer, publisher	Portfolio-level comparisons
Taxonomy	genres, categories, tags	Multi-label exploded dimensions
Reception	positive_ratings, negative_ratings	Positive rate, volume, Wilson bound
Commercial	price, owners	Price bands and reach proxies
Engagement	average_playtime, median_playtime	Descriptive engagement proxies

Research questions

Q1 How did catalogue growth and review outcomes change over time? Q2 Which genres combine breadth, attention and reliable satisfaction? Q3 How do price bands differ? Q4 Which games remain top after accounting for sample uncertainty? Q5 Which publishers combine scale and consistency? Q6 How does platform coverage vary? Q7 Which SteamSpy tags receive the most association votes?

2. HDFS storage and access strategy

The design separates immutable source files, curated columnar data and small analytical outputs. This preserves lineage, makes reruns idempotent, and avoids repeatedly parsing CSV. HDFS replication provides fault tolerance; Parquet column pruning, predicate pushdown and year partitioning reduce scan cost. The dataset is modest, so the value here is architectural correctness and scalability rather than claiming that 50 MB requires a cluster.

Kaggle CSV	HDFS RAW	PYSPARK ETL	PARQUET CURATED	SQL / FIGURES
steam.csv tag data	/projects/steam/raw immutable	schema + DQ derived fields	partitioned by release_year	CSV results PDF report

Access policy and physical layout

Layer	Path	Format	Policy
Raw	/projects/steam/raw	CSV	Immutable source; retain headers and original encoding
Curated	/projects/steam/curated/games	Parquet	Overwrite atomically; partition by release_year
Checkpoint	/projects/steam/checkpoints	Spark	Reserved for recoverable iterative/stream jobs
Results	output/results/spark	CSV	Small, human-readable aggregates for submission

Operational commands

```
hdfs dfs -mkdir -p /projects/steam/{raw,curated,checkpoints}
hdfs dfs -put -f data/raw/steam.csv /projects/steam/raw/steam.csv
spark-submit --master yarn src/steam_analysis.py --input /projects/steam/raw/steam.csv --curated
/projects/steam/curated/games --output output/results/spark
```

A real production deployment should choose Parquet file sizes near 128-256 MB and compact small files. For this snapshot, coalescing analytical outputs to one file improves demonstrability; the curated layer remains distributed. Credentials are not embedded, and the original CSV is never mutated.

3. Preprocessing and data quality

Check	Observed	Treatment
Raw rows	27,075	Schema-driven read
Duplicate app IDs	0	Drop duplicate primary keys
Missing release date	0	Retain; exclude only from time questions
Missing/negative price	0	Invalid negatives become null
Zero-review games	0	Rating and Wilson score remain null
Multi-label fields	genres/platforms/tags	Split on semicolon, trim, explode, de-duplicate

The pipeline declares explicit Spark types instead of relying on inference; trims string fields; parses ISO dates; clamps negative review counts to zero; rejects negative prices; creates `total_reviews` and `rating_pct` only where the denominator is positive; extracts owner-range endpoints; and adds a missing-core quality flag. Crucially, valid zeros are preserved. Missing publishers are not filled with a misleading category, and missing dates are handled question-by-question rather than deleting entire records.

Derived measures

Measure	Definition	Reason
<code>total_reviews</code>	positive + negative	Attention / evidence volume
<code>rating_pct</code>	$100 \times \text{positive} / \text{total}$	Interpretable game-level reception
Wilson score	95% lower confidence bound	Uncertainty-aware ranking
<code>owner_midpoint</code>	$(\text{range lower} + \text{upper}) / 2$	Approximate reach; not exact sales
<code>price_band</code>	Free, <5, 5-10, 10-20, 20-40, 40+	Robust, communicable comparison

Validation is intentionally redundant: the PySpark implementation writes a data-quality table, and an independent Pandas reference runner reproduces all submitted numerical results. This is an authoring-time cross-check, not a replacement for the cluster pipeline.

4. Analytical methods

Question	Spark method	Output
Q1 Time	year(), groupBy, count/sum/avg	Yearly release and reception series
Q2 Genre	split + explode + distinct; HAVING n>=100	Breadth, volume, mean and Wilson quality
Q3 Price	when bands; percentile_approx	Band counts, mean rating, median review volume
Q4 Ranking	UDF Wilson lower bound; threshold n>=1,000	Reliable top 20
Q5 Publisher	groupBy + scale thresholds	Consistent portfolios, not one-hit wonders
Q6 Platform	explode platforms	Coverage and mean reception
Q7 Tags	wide-to-long sparse normalization	Tag coverage, votes and mean reception
Era depth	Window + dense_rank partitioned by year	Top three games within each release cohort

Why Wilson rather than raw percentage?

Raw 100% positive means very different things for 3 reviews and 30,000 reviews. For positive count p , sample size n and $z=1.96$, the Wilson lower bound estimates a conservative plausible positive share. Its penalty shrinks as n grows. The project also requires at least 1,000 outcomes for the headline ranking; this makes both the rule and its trade-off transparent.

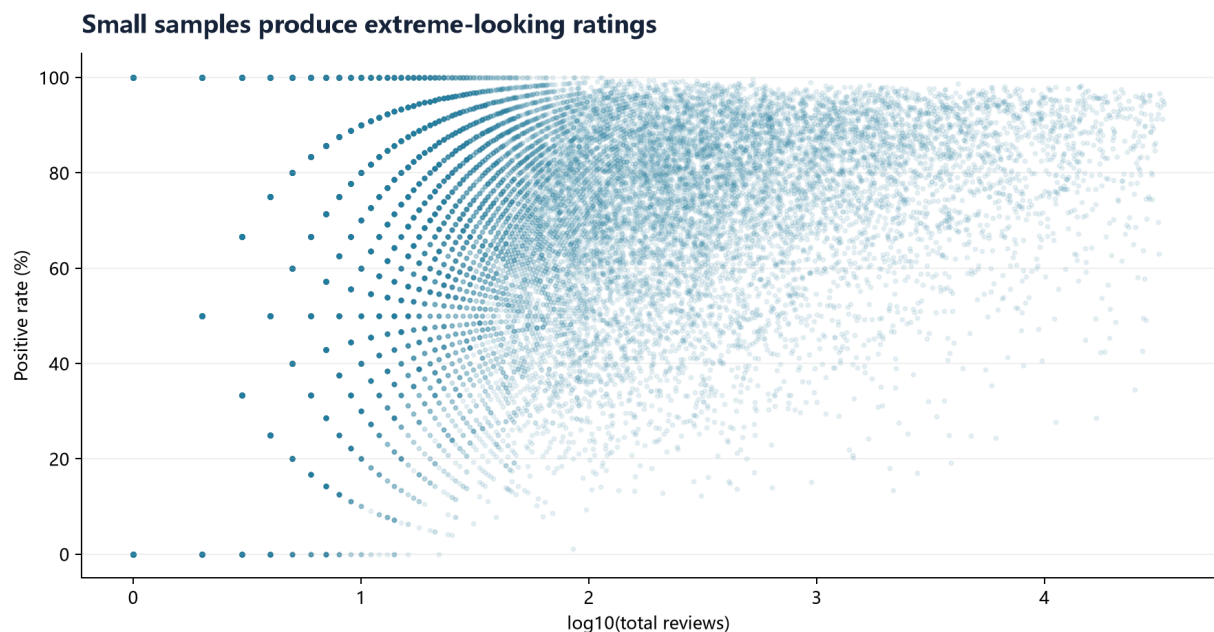


Figure 1. Rating extremes contract as evidence volume increases. Each point is one game; the x-axis is logarithmic.

All reported associations are descriptive. No regression coefficient or grouped mean is interpreted as a causal effect because price, visibility, release era, genre, discounting and audience self-selection are not randomized. This restraint is part of the analysis, not a missing flourish.

5. Findings and interpretation

5.1 Catalogue growth changed the denominator

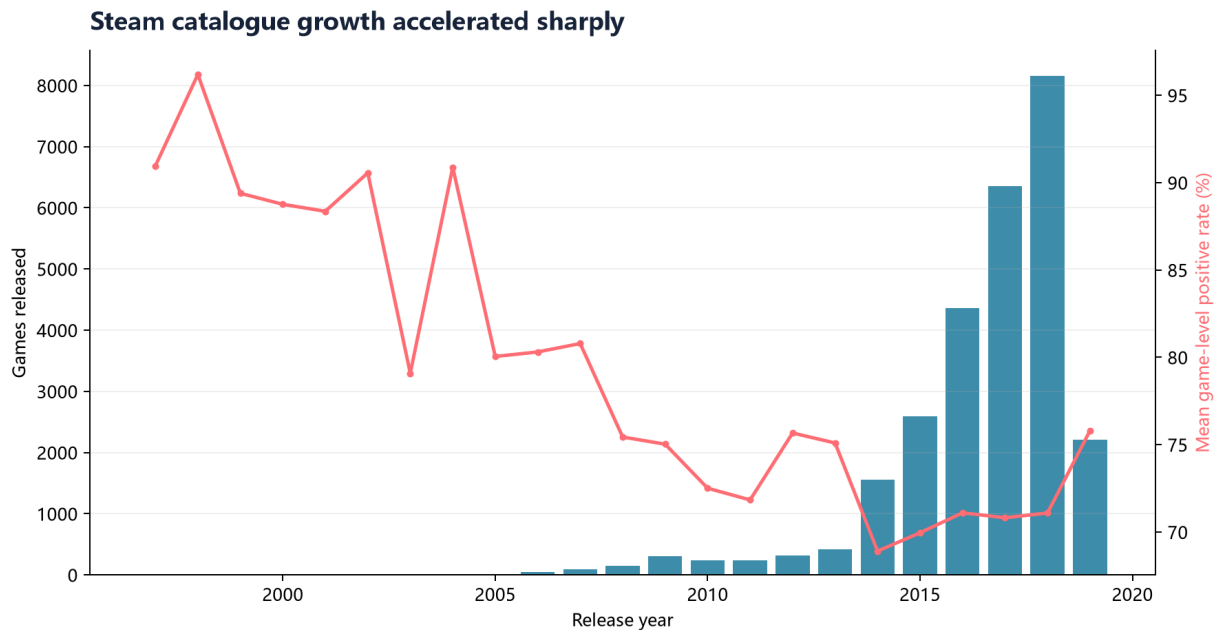


Figure 2. Releases increased rapidly after 2013; the 2019 count is incomplete because the source snapshot ends during that year.

The maximum complete annual count is 8,160 games in 2018. The sharp rise reflects Steam's expanding catalogue and lower publishing barriers, but it also changes the composition of the average: later years contain many more small-audience titles. The apparent rebound in 2019's mean rating should not be treated as a full-year trend because only part of 2019 is observed.

5.2 Weighted reception and a typical game's reception diverge

The aggregate weighted rate is 82.58%, whereas the median game is 76.03% positive and has only 36 outcomes. The weighted statistic answers 'what fraction of all recorded outcomes are positive?' while the median answers 'what does a typical listed game look like?' Reporting both prevents popularity from silently becoming a quality weight.

5.3 Genre combines scale, attention and reliability

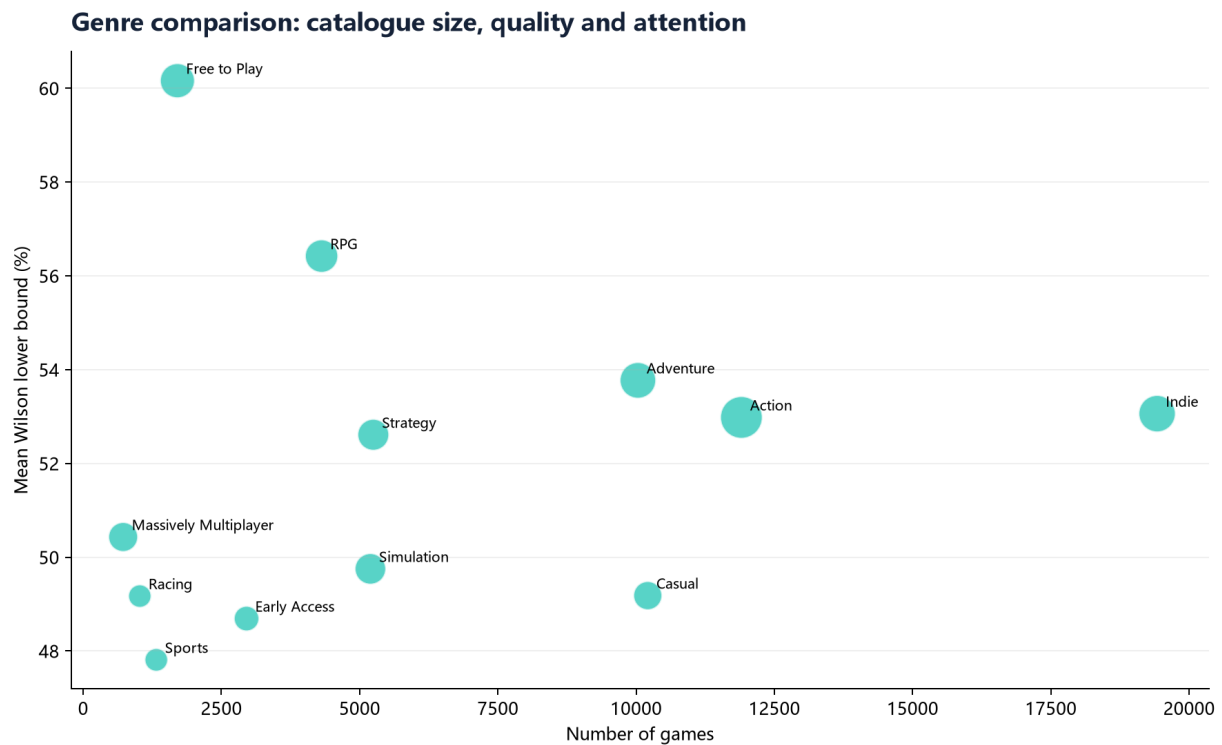


Figure 3. Bubble area represents total review outcomes. Genres overlap because games may carry multiple labels.

Genre	Games	Reviews	Mean +%	Mean Wilson %
Free to Play	1,704	8,941,644	69.95	60.16
Sexual Content	245	175,111	73.02	58.27
Nudity	266	275,115	70.91	56.85
RPG	4,311	7,005,218	71.63	56.42
Adventure	10,032	11,063,037	71.38	53.77
Indie	19,421	12,265,253	72.15	53.06
Action	11,903	23,076,274	70.99	52.98
Strategy	5,247	5,478,947	69.29	52.61

Among genres with at least 100 games, Free to Play has the highest mean Wilson lower bound in this game-level averaging scheme, but it also has a lower raw mean positive rate than several paid-oriented segments. That difference is possible because Wilson penalizes sparse evidence, and genre labels overlap. Action has the largest review volume, while Indie has the largest catalogue, so 'most common', 'most discussed' and 'most reliably positive' are three separate claims.

Interpretation caveat: the dataset's genres are multi-label and publisher-supplied. Exploding labels means one game contributes to multiple groups; totals should never be summed across genres.

5.4 Price is associated with reception, but not monotonically

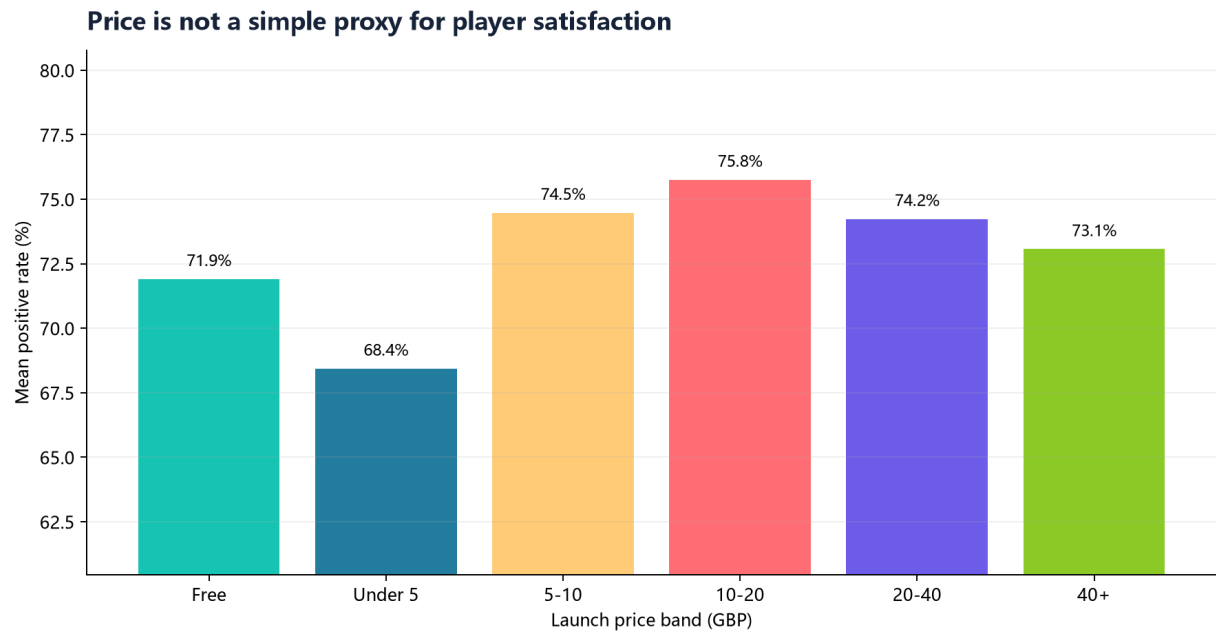


Figure 4. Game-level mean positive rates by listed price band (GBP). Values are descriptive, not causal.

Price band	Games	Mean price	Mean +%	Median reviews
Free	2,560	GBP 0.00	71.91	87
Under 5	13,286	GBP 2.63	68.44	21
5-10	6,303	GBP 7.08	74.48	36
10-20	4,001	GBP 13.62	75.76	121
20-40	810	GBP 28.28	74.24	471
40+	115	GBP 65.85	73.07	463

The highest mean game-level positive rate is 75.76% in the 10-20 band. The cheapest paid band performs worst on this measure, while GBP 40+ does not lead despite much higher median attention. This is consistent with selection and portfolio mix, not proof that changing price would change ratings. Listed price also ignores historical discounts and regional pricing.

5.5 Reliable rankings differ from naive 100% lists

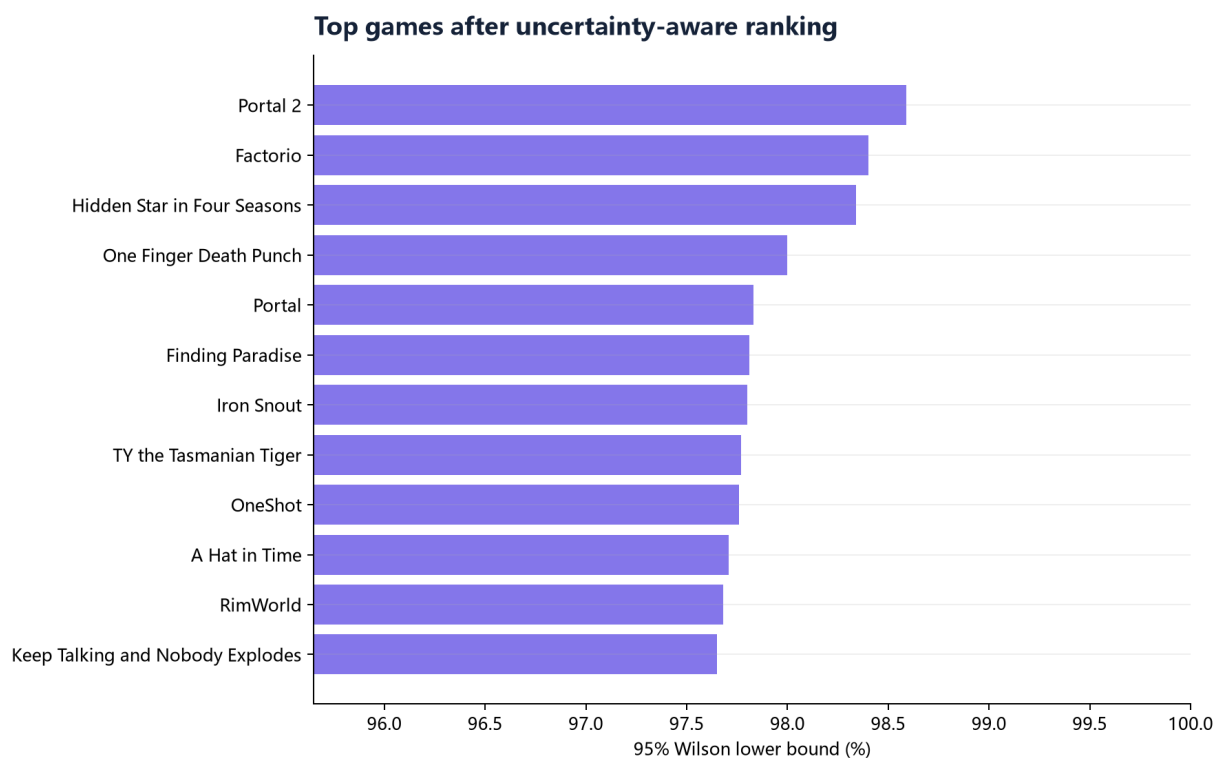


Figure 5. Top games with at least 1,000 review outcomes, ranked by 95% Wilson lower bound.

#	Game	Reviews	Positive %	Wilson %
1	Portal 2	140,111	98.65	98.59
2	Factorio	48,641	98.51	98.40
3	Hidden Star in Four Seasons	4,167	98.73	98.34
4	One Finger Death Punch	14,437	98.23	98.00
5	Portal	52,881	97.96	97.83
6	Finding Paradise	6,701	98.16	97.81
7	Iron Snout	14,166	98.04	97.80
8	TY the Tasmanian Tiger	1,702	98.47	97.77
9	OneShot	11,205	98.04	97.76
10	A Hat in Time	14,311	97.96	97.71

Portal 2 leads with 140,111 outcomes, a 98.65% raw positive rate and a 98.59% lower bound. Factorio's raw percentage is close, but its smaller sample produces a slightly larger uncertainty penalty. This ranking is reproducible, conservative and far less vulnerable to tiny-sample noise than sorting raw percentages.

5.6 Publisher portfolios

Publisher	Games	Review outcomes	Mean Wilson %
Valve	30	5,910,808	88.86
CAPCOM Co., Ltd.	13	209,151	86.31
Spike Chunsoft Co., Ltd.	15	27,421	85.64
Quiet River	21	26,456	83.85
XSEED Games;Marvelous USA, Inc.	18	13,352	83.08
id Software	19	19,075	82.75
XSEED Games;Marvelous USA, Inc.;Marvelous	10	14,342	81.73
Devolver Digital	65	327,329	81.45
MangaGamer	42	11,684	81.23
Idea Factory International	15	18,416	81.15

Valve leads among portfolios meeting both thresholds (at least 10 games and 10,000 outcomes). The rules exclude one-game publishers and limit winner's curse, but mean Wilson scores still give each game equal weight. A review-weighted portfolio score would answer a different question and would be dominated by blockbusters. Publisher strings also contain inconsistent joint-publisher combinations that should be normalized in future work.

5.7 SteamSpy tag attention

Tag	Tagged games	Tag votes	Mean +%
Action	13,136	763,849	71.20
Indie	20,782	730,458	72.19
Adventure	11,572	563,927	71.86
Multiplayer	3,111	415,341	73.86
Singleplayer	6,779	403,390	76.39
Casual	11,449	327,532	71.50
Rpg	4,900	318,176	71.97
Strategy	6,000	312,526	69.74

Tag votes measure how strongly SteamSpy users associate games with descriptors, not review sentiment. The Spark pipeline converts the 370+ wide vote columns into a sparse (appid, tag, votes) relation, then aggregates only positive votes. This demonstrates scalable wide-to-long normalization without collecting the vote matrix on the driver.

6. Limitations and threats to validity

Limitation	Consequence / mitigation
Static snapshot around May 2019	No current-market claims; 2019 is incomplete
Aggregate outcomes, no text or users	No NLP, reviewer behavior or demographic inference
SteamSpy owner ranges	Use midpoint only as an approximate reach proxy
Listed price only	No discount history, region or bundle adjustment
Multi-label, publisher-defined taxonomy	Explode carefully; do not sum overlapping genres
Survivorship/visibility selection	Dataset is not a randomized sample of all games or players
Observational comparisons	Describe associations; avoid causal language

How the design responds

These limitations are handled by narrowing claims rather than hiding uncertainty. Time analysis explicitly marks 2019 as incomplete; owner counts are labeled as range-derived proxies; genre totals are never added across overlapping labels; and the report consistently uses association language. Wilson bounds address sampling uncertainty in rankings, but they cannot repair selection bias or omitted variables.

External validity is bounded to the dataset snapshot. Internal reproducibility is stronger: raw files are immutable, transformations are deterministic, every result is machine-readable, and two implementations share the same explicit formulas. The remaining cluster-specific check is operational execution on the course Hadoop environment, because this authoring machine does not provide HDFS or Java/Spark services.

7. Conclusion and critical reflection

The project demonstrates a complete big-data workflow: immutable HDFS ingestion, explicit-schema PySpark cleaning, columnar curation, Spark SQL analysis, multi-label normalization, statistical ranking, and data-driven visualization. The strongest substantive result is not a single winning genre or price. It is that aggregation choices materially change the story: weighted reception is higher than a typical game's reception, popularity is not quality, and raw percentages are unreliable when evidence is sparse.

The most important design lesson was to match the statistic to the question. A global weighted mean describes all review outcomes; a game-level median describes a typical title; a Wilson lower bound supports robust ranking; and price-band means describe market segments without identifying a price effect. Building all four from the same curated table makes those distinctions testable rather than rhetorical.

Future work

A stronger next version would ingest dated individual reviews from a separate Steam review source, perform language-aware sentiment/topic modeling, model time-to-review and update effects, normalize publisher entities, and add discount histories. Incremental ingestion could be partitioned by snapshot date with data-quality tests in CI. The included Streamlit dashboard can be deployed to the course server; a future version could add authenticated drill-down while continuing to serve only pre-aggregated data.

Team roles

Member	Primary responsibility	Shared responsibility
Student 1	HDFS ingestion, PySpark ETL, data quality validation	Research design, review, validation and presentation
Student 2	Spark SQL analysis, visualization, interpretation and report	Research design, review, validation and presentation

Both members jointly reviewed analytical assumptions, reran the reproducibility workflow, checked tables against machine-readable outputs, and approved the final interpretation. Replace placeholder names in project_metadata.json before submission.

8. Reproducibility and submission map

Reproduction sequence

1. Download/extract the assignment-specified Kaggle dataset to data/raw.
2. Run scripts/hdfs_setup.ps1 to create the HDFS raw layer.
3. Run scripts/run_pipeline.ps1 to build Parquet and Spark results.
4. Run tools/local_reference_analysis.py to cross-check metrics and figures.
5. Run tools/build_report.py; inspect the rendered pages before submission.

Code and evidence inventory

Artifact	Purpose
src/steam_analysis.py	Executable PySpark ETL, quality checks, SQL and window ranking
sql/steam_analysis.sql	Standalone Spark SQL / HiveQL analytical queries
scripts/hdfs_setup.ps1	Idempotent HDFS directory creation and raw ingestion
scripts/run_pipeline.ps1	YARN spark-submit entry point and Spark configuration
output/results/local	CSV/JSON evidence used by this report
output/figures	High-resolution visualizations
dashboard/app.py	Optional interactive bonus dashboard over pre-aggregated results
README.md	Environment, commands, dataset and project structure

Rubric coverage

Rubric item	Evidence
Dataset introduction	Section 1 and data dictionary
HDFS strategy	Section 2, ingestion script, raw/curated separation
Cleaning/parsing/missing values	Section 3 and explicit PySpark schema
Questions and Spark/SQL methods	Sections 4-5 and SQL script
Findings and visualization	Five interpreted figures and result tables
Critical conclusion/reflection	Sections 6-7
Executable, styled source	Modular scripts, comments, requirements and README
Method depth	Wilson bound, scale thresholds, windowed era ranking, cross-check

References

[1] Davis, N. (2019). *Steam Store Games*. Kaggle.

<https://www.kaggle.com/datasets/nikdavis/steam-store-games/data>

[2] Apache Software Foundation. *Apache Hadoop HDFS Architecture*.

<https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>

[3] Apache Software Foundation. *Apache Spark SQL, DataFrames and Datasets Guide*.

<https://spark.apache.org/docs/latest/sql-programming-guide.html>

[4] Apache Software Foundation. *PySpark API Reference*. <https://spark.apache.org/docs/latest/api/python/>

[5] Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158), 209-212.

[6] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95.

Result provenance: every number and figure in this report was generated from `data/raw/steam.csv` by `tools/local_reference_analysis.py`. The assessed distributed implementation mirrors the same transformations in `src/steam_analysis.py`. No result was manually typed into the report generator.